

Совместная программа СМОЛ ГУ и
ФКН НИУ ВШЭ

2026/27 учебный год

Методы оптимизации в машинном обучении

Лекция 7. Матричное и автоматическое дифференцирование

Никита Лукьяненко

Градиентный спуск: откуда берётся градиент?

За шесть лекций мы научились исследовать метод

$$\boxed{x_{k+1} = x_k - \alpha_k \nabla f(x_k).}$$

Мы обсуждали кривизну, число обусловленности и скорость сходимости.

Скрытая вычислительная задача

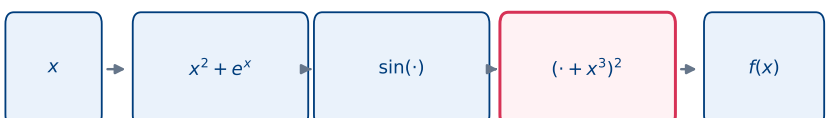
Каждая итерация требует вычислить $\nabla f(x_k)$. Что делать, если f включает тысячи параметров и промежуточных операций?

Функцию просто вычислить, но сложно дифференцировать

Для $f(x) = x^2 + \sin x$ производная очевидна. Но даже композиция

$$f(x) = \left(\sin(x^2 + e^x) + x^3\right)^2$$

выглядит намного менее дружелюбно после полного раскрытия производной.



Идея: дифференцировать элементарные операции по отдельности.

Автоматическое дифференцирование: точное определение

Определение

Автоматическое дифференцирование (АД) — это вычисление производных функции, заданной последовательностью элементарных операций, посредством систематического применения **правила цепочки** к каждой операции.

- **Символьный подход:** строим формулу производной как математическое выражение.
- **Численный подход:** приближаем производную конечными разностями с шагом h.
- **Автоматический подход:** на конкретных входных данных вычисляем значения операций и их производные по точным аналитическим правилам.

Важно

АД не использует конечные разности и не требует раскрывать полную символическую формулу. Точность всё равно ограничена машинной арифметикой.



Карта сегодняшней лекции

$$\boxed{\text{Дифференциал}} \Longrightarrow \boxed{\text{Правило цепочки}}$$

$$\boxed{\text{Вычислительный граф}} \Longrightarrow \boxed{\text{Автоматический градиент}}.$$

В конце лекции самостоятельно пройдём по графу функции

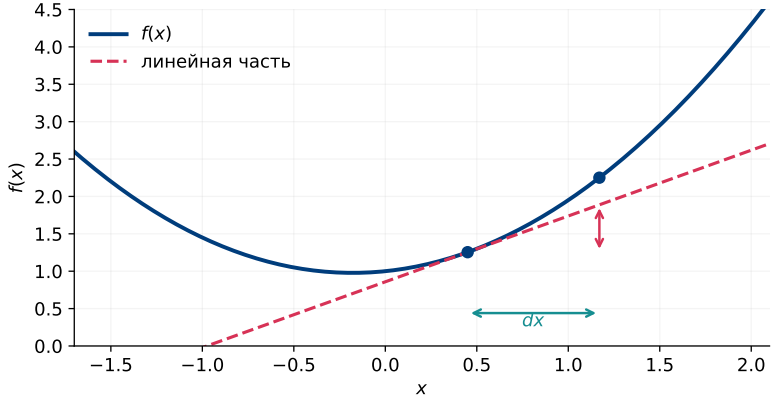
$$\boxed{f(x, y) = (xy + \sin x)^2}$$

и получим производные в двух разных режимах.

Возвращаемся к линейному приближению

Из лекции №2 для дифференцируемой $f : \mathbb{R}^d \to \mathbb{R}$:

$$f(x + h) = f(x) + \nabla f(x)^\top h + o(\|h\|).$$



Дифференциал — главная линейная часть изменения

Приращение dx может быть малым вектором произвольного направления. Определяем дифференциал как линейную по dx часть приращения:

$$\boxed{df = \nabla f(x)^\top dx.}$$

Важно различать:

$$f(x + dx) - f(x) = df + o(\|dx\|).$$

Не путать

df не обязано совпадать с настоящим конечным приращением функции. Совпадает лишь его линейная часть.

Пример: как прочитать градиент из дифференциала

Пусть $f(x, y) = x^2 + 3xy$. Тогда

$$df = (2x + 3y)\,dx + 3x\,dy.$$

Соберём коэффициенты в столбец:

$$df = \underbrace{\begin{pmatrix} 2x + 3y & 3x \end{pmatrix}}_{\nabla f(x,y)^\top} \begin{pmatrix} dx \\ dy \end{pmatrix}.$$

Рабочий приём

Привести df к форме $g^\top dx$, после чего прочитать $\nabla f = g$.



А если результат — целый вектор?

Пусть $g : \mathbb{R}^d \to \mathbb{R}^m$, тогда дифференциал тоже является вектором:

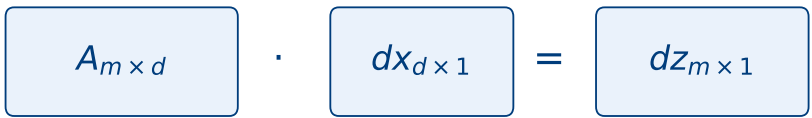
$$dg = J_g(x)\, dx,$$

где $J_g(x)$ — матрица линейного приближения.

$$\underbrace{dg}_{m \times 1} = \underbrace{J_g(x)}_{m \times d} \underbrace{dx}_{d \times 1}.$$

Определение матрицы Якоби подробно появится чуть позже.

Проверка размерностей — лучший способ найти ошибку



Для любого дифференцируемого отображения $g : \mathbb{R}^d \to \mathbb{R}^m$ размерность якобиана определяется автоматически: $m \times d$.

Правило

Если произведение матриц не определено или результат имеет неверную размерность, формула производной записана неправильно.

Аффинное преобразование: $z = Ax + b$

Возьмём $A \in \mathbb{R}^{m \times d}$, $x \in \mathbb{R}^d$. Тогда

$$z(x + dx) - z(x) = A(x + dx) + b - (Ax + b) = A\,dx.$$

Следовательно,

$$\boxed{dz = A\,dx, \qquad J_z(x) = A.}$$

Матрица A и есть линейная часть изменения z. Никаких координатных сумм не потребовалось.

Квадратичная потеря: разделим вычисление на операции

Пусть

$$\boxed{f(x) = \frac{1}{2}\|Ax - b\|_2^2.}$$

Введём промежуточный остаток $r = Ax - b$, тогда

$$f(x) = \frac{1}{2} r^\top r.$$

Теперь достаточно продифференцировать **две простые операции**:

$$x \longmapsto r, \qquad r \longmapsto \tfrac{1}{2} r^\top r.$$

Дифференциал квадрата нормы

Для $r \in \mathbb{R}^m$ раскрываем квадрат приращения:

$$\tfrac{1}{2}(r + dr)^\top (r + dr) = \tfrac{1}{2} r^\top r + r^\top dr + \tfrac{1}{2} \|dr\|^2 .$$

Линейная часть равна

$$\boxed{df = r^\top dr.}$$

Для $r = Ax - b$ уже известно

$$\boxed{dr = A\,dx.}$$

Первое доказательство: градиент квадратичной потери

Утверждение

Для $f(x) = \frac{1}{2}\,||Ax - b||_2^2$ выполнено

$$\boxed{\nabla f(x) = A^\top (Ax - b).}$$

Доказательство:

$$\begin{aligned} df &= r^\top dr = r^\top A\, dx \\ &= (A^\top r)^\top dx. \end{aligned}$$

Сопоставляя с $df = \nabla f(x)^\top dx$, получаем требуемое. $\square$

От ошибки одного объекта — к эмпирическому риску

Даны наблюдения (x_i, y_i), $i = 1, \dots, n$, и линейный прогноз $\hat{y}_i = w^\top x_i$. Для одного объекта возьмём квадратичную потерю

$$\ell_i(w) = \frac{1}{2}(w^\top x_i - y_i)^2.$$

Эмпирический риск

Средняя потеря *на наблюдённой выборке*:

$$\boxed{\widehat{R}_n(w) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(w).}$$

В отличие от теоретического риска $R(w) = \mathbb{E}\,\ell(w; (X, Y))$ — среднего по неизвестному распределению данных — здесь усредняем только имеющиеся наблюдения.

Как отдельные ошибки влияют на каждый параметр

Обозначим ошибку предсказания на объекте i через

$$r_i = w^\top x_i - y_i.$$

Тогда градиент отдельной потери:

$$\nabla \ell_i(w) = r_i x_i.$$

И градиент эмпирического риска — среднее **векторов**:

$$\boxed{\nabla \widehat{R}_n(w) = \frac{1}{n} \sum_{i=1}^{n} r_i x_i.}$$

Каждая ошибка r_i умножается на вектор признаков x_i: так она даёт вклад в **каждую координату** градиента.

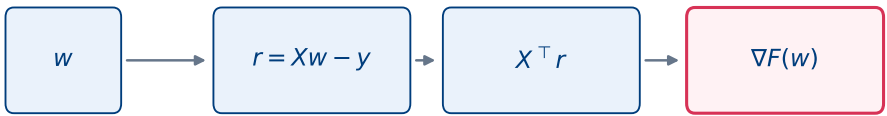
Матричная запись градиента эмпирического риска

В матрице $X \in \mathbb{R}^{n \times d}$ строки — это векторы признаков $x_i^\top$. Остатки собираются в столбец

$$r = Xw - y \in \mathbb{R}^n.$$

Поэтому средний риск и его градиент записываются компактно:

$$\widehat{R}_n(w) = \frac{1}{2n} \|Xw - y\|_2^2, \qquad \boxed{\nabla \widehat{R}_n(w) = \frac{1}{n} X^\top r.}$$



$X^\top$ переводит n ошибок объектов в d компонент градиента; $1/n$ усредняет по выборке.

Добавим регуляризацию

Рассмотрим

$$F_\lambda(w) = \frac{1}{2n} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2 .$$

Из линейности дифференциала:

$$\boxed{\nabla F_\lambda(w) = \frac{1}{n} X^\top (Xw - y) + \lambda w.}$$

Вклад штрафа прибавляется к уже найденному градиенту основной потери.



Логистическая потеря: производная по одному аргументу

Пусть

$$\ell(z) = \log(1 + e^{-z}).$$

Обычное правило цепочки:

$$\ell'(z) = \frac{-e^{-z}}{1 + e^{-z}} = \boxed{-\frac{1}{1 + e^{z}}}.$$

Это локальная производная **по скалярному** аргументу z.



Логистическая потеря: откуда берётся аргумент

Для наблюдения (x_i, y_i) с $y_i \in \{-1, 1\}$:

$$z = y_i w^\top x_i, \quad \ell_i(w) = \log(1 + e^{-z}).$$

Функция устроена как композиция:

$$w \longmapsto z \longmapsto \ell(z).$$

Поскольку z линейна по w,

$$\boxed{\nabla_w z = y_i x_i.}$$



Градиент логистической потери

По правилу цепочки:

$$\nabla_w \ell_i(w) = \ell'(z) \nabla_w z.$$

Подставим обе уже найденные производные:

$$\boxed{\nabla_w \ell_i(w) = -\frac{y_i x_i}{1 + e^{y_i w^\top x_i}}.}$$

Никакой новой сложной техники здесь нет: только композиция двух функций.

Общая структура: от параметров до потери

В рассмотренных моделях вычисление имеет вид

$$\boxed{\text{параметры}} \longrightarrow \boxed{\text{прогноз}} \longrightarrow \boxed{\text{потеря}} \longrightarrow \boxed{\text{средний риск}}.$$

Чтобы продифференцировать весь эмпирический риск, достаточно знать производную **каждой отдельной операции** и правило их композиции.

Следующая цель

Найти универсальное правило для отображений между пространствами *разных размерностей*.



Привычное правило цепочки

Для одномерных функций:

$$\boxed{(h \circ g)'(x) = h'(g(x))g'(x).}$$

Например, для $f(x, y) = \sin(xy)$ вначале вычисляем $g(x, y) = xy$, затем $h(z) = \sin z$.

Вопрос

А если промежуточный результат — **вектор**, а не число?

Один параметр может влиять по двум путям

Введём

$$a = xy, \qquad b = \sin x, \qquad f = a + b.$$

Переменная x влияет на f и через a, и через b:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial x} = y + \cos x.$$

Эти два вклада необходимо **сложить**.



Определение матрицы Якоби

Определение

Для дифференцируемой $g : \mathbb{R}^d \to \mathbb{R}^m$ её матрица Якоби — матрица всех первых частных производных:

$$J_g(x) = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \cdots & \frac{\partial g_m}{\partial x_d} \end{pmatrix}.$$

У матрицы m строк (по выходам) и d столбцов (по входам).

Пример вычисления матрицы Якоби

Пусть

$$g(x, y) = \begin{pmatrix} xy \\ \sin x \end{pmatrix}.$$

Производные первой компоненты: y, x; второй: $\cos x$, 0. Итак,

$$\boxed{J_g(x, y) = \begin{pmatrix} y & x \\ \cos x & 0 \end{pmatrix}.}$$

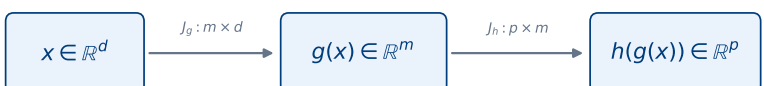
Строка отвечает за одну **выходную** координату.

Теорема: многомерное правило цепочки

Теорема

Пусть $g : \mathbb{R}^d \to \mathbb{R}^m$ дифференцируема в x, а $h : \mathbb{R}^m \to \mathbb{R}^p$ дифференцируема в $g(x)$. Тогда

$$\boxed{J_{h \circ g}(x) = J_h(g(x)) J_g(x).}$$



Доказательство правила цепочки: два линейных приближения

Запишем композицию как $x \mapsto z = g(x) \mapsto h(z)$. Если аргумент меняется на Δx, промежуточное значение меняется на

$$\Delta z = g(x + \Delta x) - g(x) = J_g(x)\Delta x + r_g(\Delta x),$$

где $r_g(\Delta x) = o(\|\Delta x\|)$. Для второй функции:

$$h(z + \Delta z) - h(z) = J_h(z)\Delta z + r_h(\Delta z),$$

где $r_h(\Delta z) = o(\|\Delta z\|)$.

Следующий шаг

Подставим первое приближение в другое и соберём коэффициент при Δx.



Доказательство правила цепочки: подстановка

Подставим $\Delta z = J_g(x)\Delta x + r_g(\Delta x)$:

$$\begin{aligned}\Delta(h \circ g) &= J_h(g(x))J_g(x)\Delta x \\ &\quad + J_h(g(x))r_g(\Delta x) + r_h(\Delta z).\end{aligned}$$

Поскольку $\Delta z = O(\|\Delta x\|)$, оба остаточных члена составляют $o(\|\Delta x\|)$. Значит,

$$\boxed{J_{h\circ g}(x) = J_h(g(x))J_g(x).}$$

Проверка размерностей

$(p \times m)(m \times d) = p \times d$: промежуточное пространство $\mathbb{R}^m$ «сокращается».

Для скалярного выхода правило становится проще

Если $h : \mathbb{R}^m \to \mathbb{R}$, то $J_h = \nabla h^\top$ — строка. Получаем

$$J_{h \circ g}(x) = \nabla h(g(x))^\top J_g(x).$$

Транспонируем, чтобы получить градиент-столбец:

$$\boxed{\nabla (h \circ g)(x) = J_g(x)^\top \nabla h(g(x)).}$$

Этот порядок матричного произведения ляжет в основу обратного режима автоматического дифференцирования.

Применение правила цепочки к знакомой функции

Пусть

$$g(x,y) = \begin{pmatrix} xy \\ \sin x \end{pmatrix}, \qquad h(u,v) = (u+v)^2.$$

Тогда

$$\nabla h(u,v) = 2(u+v)\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad J_g(x,y)^\top = \begin{pmatrix} y & \cos x \\ x & 0 \end{pmatrix}.$$

Следовательно,

$$\boxed{\nabla(h \circ g)(x,y) = 2(xy + \sin x)\begin{pmatrix} y + \cos x \\ x \end{pmatrix}.}$$



Зададим сложную функцию как последовательность операций

Будем исследовать одну функцию до конца лекции:

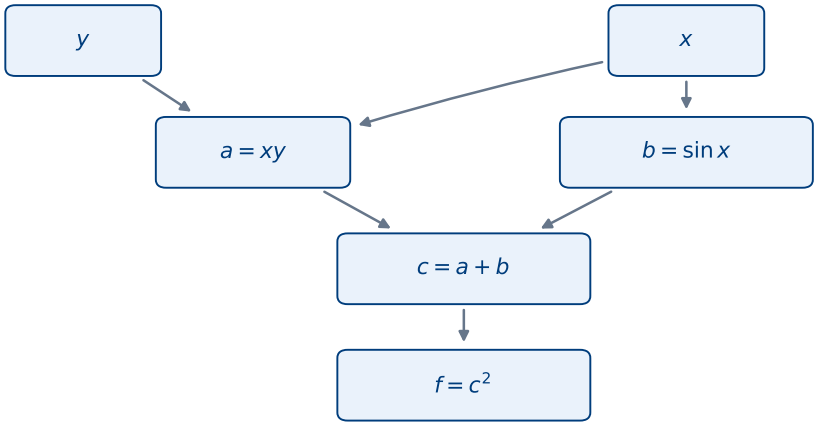
$$\boxed{f(x, y) = (xy + \sin x)^2.}$$

Введём промежуточные значения

$$a = xy, \qquad b = \sin x, \qquad c = a + b, \qquad f = c^2.$$

Вычислительный граф хранит **зависимости** между этими операциями.

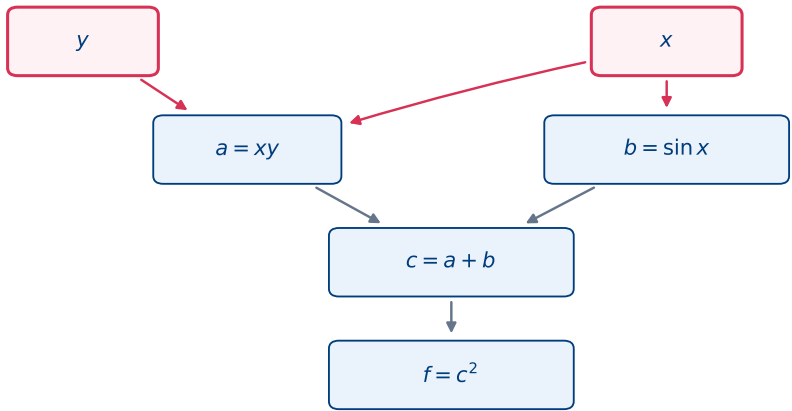
Вычислительный граф



Узлы — результаты элементарных операций, стрелки — зависимости. Порядок вычислений определяется направлением стрелок.

Прямой проход: начальные значения

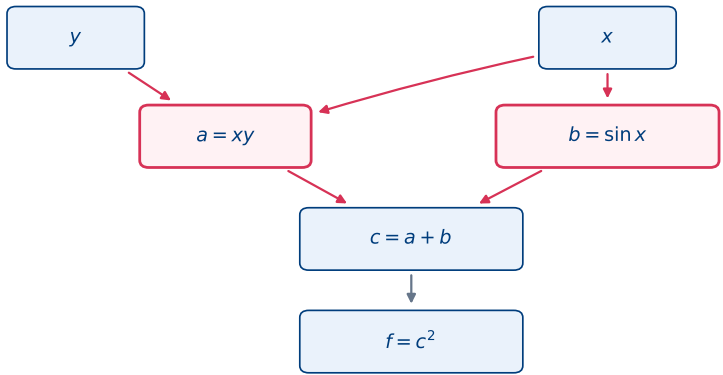
Пусть $x = 1$, $y = 2$. Сначала значения известны только во входных узлах.



Прямой проход: промежуточные вычисления

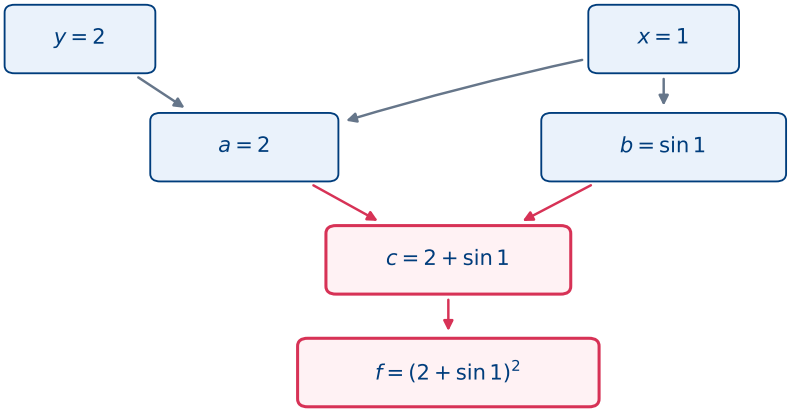
Следующий уровень можно вычислить независимо:

$$a = xy = 2, \qquad b = \sin x = \sin 1.$$



Прямой проход: вычисляем конечный результат

Теперь $c = 2 + \sin 1$, затем $f = (2 + \sin 1)^2$.





Локальные производные в каждом узле

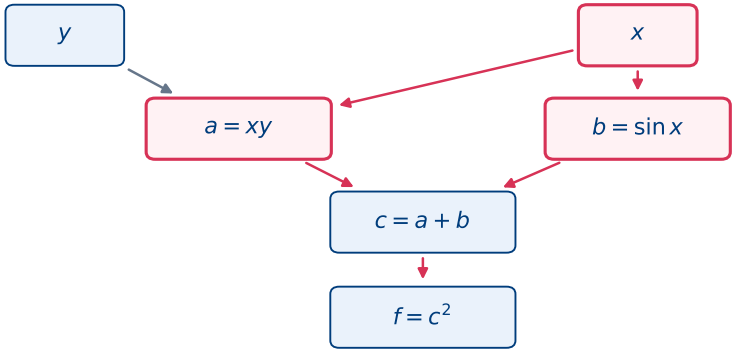
Почти все необходимые производные элементарны:

$$
\begin{aligned}
a = xy: &\qquad a_x = y, \quad a_y = x,\\
b = \sin x: &\qquad b_x = \cos x,\\
c = a + b: &\qquad c_a = c_b = 1,\\
f = c^2: &\qquad f_c = 2c.
\end{aligned}
$$

Наблюдение

Вычислить *одну локальную производную* гораздо проще, чем сразу получить градиент всей длинной композиции.

Два пути влияния одного параметра



$$\frac{\partial f}{\partial x} = 2c(y + \cos x), \qquad \frac{\partial f}{\partial y} = 2cx.$$



Зачем нам два способа проходить по графу?

Пусть у функции d входных параметров, а на выходе m чисел:

$$F : \mathbb{R}^d \longrightarrow \mathbb{R}^m.$$

Нужно уметь считать, **как малое изменение входов отражается на выходах**.

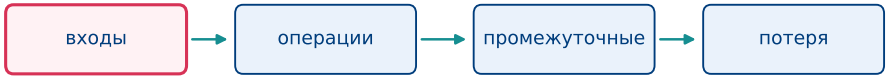
- В **прямом режиме** выбираем изменение входов v и отслеживаем, как оно проходит через операции. Получаем $J_F(x)v$.
- В **обратном режиме** выбираем скалярную комбинацию выходов $u^\top F$ и выясняем, *через какие промежуточные узлы* она зависит от входов. Получаем $J_F(x)^\top u$.

Что такое «вклад»?

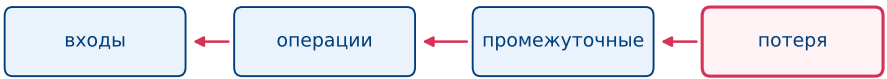
Если вход влияет на результат двумя путями, его производная равна **сумме вкладов** каждого пути по правилу цепочки.

Два режима: что распространяется по графу?

Прямой режим: значения и направленные производные



Обратный режим: производные от выхода к входам



Прямой проход: от входов к выходу — вычисляем значения и направленные производные.

Обратный проход: после вычисления значений идём от выхода к входам и накапливаем производные.



Прямой режим: что означает точка над переменной?

Вообразим, что исходные аргументы меняются с параметром t:

$$x(t) = x + tv_x, \qquad y(t) = y + tv_y.$$

Выбранное направление — это вектор $v = (v_x, v_y)$. **Точкой** обозначаем скорость изменения узла при $t = 0$:

$$\boxed{\dot{u} := \left.\frac{du(t)}{dt}\right|_{t=0}.}$$

Например, для $a(t) = x(t)y(t)$:

$$\dot{x} = v_x, \quad \dot{y} = v_y, \qquad \boxed{\dot{a} = y\dot{x} + x\dot{y}.}$$

Если хотим $\partial f / \partial x$, берём $v = (1, 0)$: меняется только x, а y остаётся постоянным.



Прямой режим: локальные правила

$$
\begin{aligned}
a = u + v \quad &\Longrightarrow \quad \dot{a} = \dot{u} + \dot{v}, \\
a = uv \quad &\Longrightarrow \quad \dot{a} = v\dot{u} + u\dot{v}, \\
a = \sin u \quad &\Longrightarrow \quad \dot{a} = \cos(u)\dot{u}, \\
a = u^2 \quad &\Longrightarrow \quad \dot{a} = 2u\dot{u}.
\end{aligned}
$$

Это ровно знакомые правила дифференцирования, применяемые при обходе графа.



Прямой режим: находим $\partial f / \partial x$

Начинаем с $\dot{x} = 1$, $\dot{y} = 0$:

$$\dot{a} = y\dot{x} + x\dot{y} = y, \qquad \dot{b} = \cos x\, \dot{x} = \cos x.$$

$$\dot{c} = \dot{a} + \dot{b} = y + \cos x.$$

В конечном узле:

$$\boxed{\dot{f} = 2c\dot{c} = 2(xy + \sin x)(y + \cos x).}$$

Прямой режим: как найти другую компоненту?

Для $\partial f / \partial y$ изменяем только начальные производные:

$$\boxed{\dot{x} = 0, \qquad \dot{y} = 1.}$$

Тогда

$$\dot{a} = x, \quad \dot{b} = 0, \quad \dot{c} = x, \quad \boxed{\dot{f} = 2cx.}$$

Ограничение прямого режима

Для полного градиента скалярной функции d переменных в общем случае нужны d независимых направленных проходов.



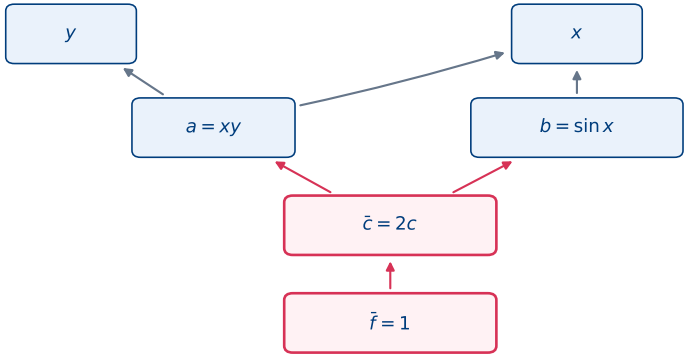
Обратный режим: поменяем направление обхода

Сначала выполняем **обычный прямой проход**, чтобы знать значения всех узлов.
Затем начинаем с конечной функции. Вводим обозначение:

$$\boxed{\bar{u} := \frac{\partial f}{\partial u}.}$$

В конечном узле всегда $\bar{f} = 1$. Остальные величины получим, проходя от выхода к входам.

Обратный проход: от результата к сумме



Так как $f = c^2$ и $\bar{f} = 1$, получаем

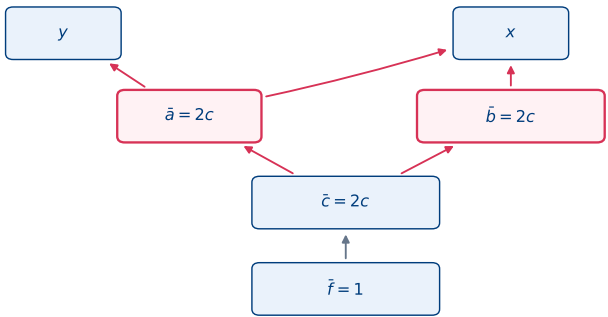
$$\boxed{\bar{c} = 2c.}$$

На схеме стрелки показывают порядок **распространения производных**, а не вычисления значений.

Обратный проход: разветвление и накопление

Поскольку $c = a + b$, от узла c получаем

$$\bar{a} = \bar{c}\frac{\partial c}{\partial a} = 2c, \qquad \bar{b} = \bar{c}\frac{\partial c}{\partial b} = 2c.$$





Обратный проход: собираем градиент по входам

От ветки $a = xy$:

$$\bar{x} \mathrel{+}= \bar{a}\, y, \qquad \bar{y} \mathrel{+}= \bar{a}\, x.$$

От ветки $b = \sin x$:

$$\bar{x} \mathrel{+}= \bar{b} \cos x.$$

В итоге

$$\boxed{\bar{x} = 2c(y + \cos x), \qquad \bar{y} = 2cx.}$$

Ключевой принцип

Если в параметр ведёт несколько путей, их вклады в производную **складываются**.



Какой режим выгоднее?

Прямой режим	**Обратный режим**
Вычисляет Jv	Вычисляет $J^\top u$
Особенно удобен при небольшом числе входов	Особенно удобен при небольшом числе выходов
Для полного градиента нужен проход по каждому базисному направлению	Для скалярного выхода достаточно одного обратного прохода

Цена обратного режима: часто нужно сохранять промежуточные значения (или пересчитывать их), то есть возникает компромисс между памятью и временем.

А можно просто приближать производные числами?

Например, центральная конечная разность:

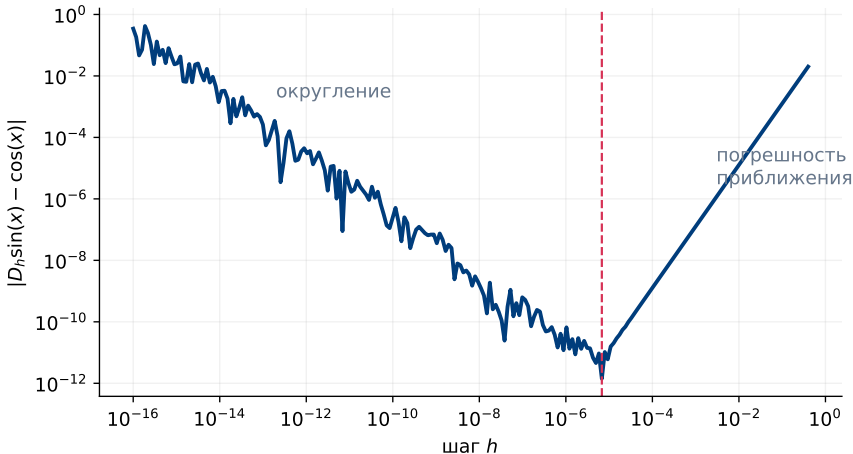
$$\boxed{f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.}$$

При достаточно гладкой функции погрешность усечения имеет порядок $O(h^2)$, но слишком малый шаг усиливает влияние ошибок округления.

Вопрос

Почему нельзя просто выбрать $h = 10^{-100}$ и получить идеальную производную?

Численная производная: большой и очень маленький шаг



При слишком большом h приближение грубо; при слишком маленьком возникают потери точности из-за вычитания близких чисел.



Как выглядит автоматическое дифференцирование в Python

```
import torch
x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(2.0, requires_grad=True)
f = (x*y + torch.sin(x))**2
f.backward()
print(x.grad.item(), y.grad.item())
```

Команда `backward()` выполняет обратный проход и заполняет производные у входных переменных.

Математика не изменилась

Компьютер применяет то же правило цепочки, которое мы только что проделали вручную.



Проверяем три способа вычислить производную

Для $f(x, y) = (xy + \sin x)^2$ можно сравнить:

- **Аналитическую:** $\partial_x f = 2(xy + \sin x)(y + \cos x)$;
- **Численную:** конечную разность по x;
- **Автоматическую:** результат обратного прохода.

Как интерпретировать согласие

Совпадение результатов — полезная проверка реализации, но не формальное доказательство правильности для всех входов.

Три частые ошибки в реальной реализации

1. **Размерности:** записали $A(Ax - b)$ вместо $A^\top(Ax - b)$.
2. **Масштаб потери:** при переходе от суммы к среднему забыли множитель $1/n$.
3. **Накопление:** повторно вызвали обратный проход, не очистив ранее накопленные градиенты в библиотеке.

Почему помогает теория

Если понимать, что именно вычисляет градиент, такие ошибки легче заметить, чем при бездумном использовании библиотеки.



Карта лекции

$$\boxed{df = J_f\, dx}$$

$$\Downarrow \quad \boxed{J_{h \circ g} = J_h J_g}$$

$$\Downarrow \quad \boxed{\text{локальные производные на графе}}$$

$$\Downarrow \quad \boxed{\text{прямой и обратный проходы}}$$

Автоматическое дифференцирование — алгоритмическое применение правила цепочки, а не численная аппроксимация конечными разностями.



Следующая лекция: как выбирать шаг?

Теперь мы знаем, **как вычислить** $\nabla f(x_k)$ даже для сложной функции.
Но градиентный спуск требует ещё одного решения:

$$\boxed{x_{k+1} = x_k - \alpha_k \nabla f(x_k).}$$

Следующий вопрос

Как выбирать α_k и когда останавливать алгоритм, если оптимальное решение заранее неизвестно?

Далее — поиск шага, достаточное убывание и критерии остановки.